Vision por Computadora

De los pixeles a los modelos de fundacion

Curso aplicado al pipeline FPM-CubeSat

Material acompañante de `CubeSat-EdgeAI-Payload`

28 de abril de 2026

Prefacio

Este libro es una guia para llevarte de cero a los modelos de vanguardia en vision por computadora, usando tu propio pipeline de microscopia FPM como hilo conductor. Cada concepto que aprendas va a tener un anclaje concreto: una funcion en `pipeline/`, un parametro en `config.yaml`, un experimento que podes correr sobre tus imagenes.

Como leer este libro

El libro tiene seis partes que avanzan en complejidad como las escalas de la astronomia: del atomo (un pixel) hasta el universo observable (modelos de fundacion).

Parte I — Fundamentos.
La imagen como dato. Operaciones que tocan pixel a pixel.

Parte II — Estructura geometrica.
Como la imagen tiene forma: bordes, ridges, regiones, contornos.

Parte III — El espacio de Fourier.
La otra mitad del mundo: frecuencias, optica, FPM.

Parte IV — Multi-imagen.
Cuando una sola foto no alcanza: registracion, super-resolucion.

Parte V — Aprendizaje profundo.
Redes convolucionales, U-Net, Cellpose, StarDist.

Parte VI — Vanguardia.
Self-supervised, foundation models, modelos generativos.

Cada capitulo tiene la misma estructura:

1. **Por que importa** — el problema concreto que resuelve.
2. **Intuicion** — la metafora antes de la formula.
3. **Matematica** — las definiciones formales y deducciones clave.
4. **Algoritmo** — el pseudocodigo paso a paso.
5. **Implementacion** — en NumPy puro y comparado con OpenCV/scikit-image.
6. **Conexion con tu pipeline** — donde aparece este concepto en tu codigo.
7. **Ejercicios** — experimentos concretos sobre tus imagenes.

Prerequisitos

Asumimos:

- Algebra lineal basica: vectores, matrices, multiplicacion, autovalores.
- Calculo de una variable: derivada, integral. Multivariable es bienvenido pero no obligatorio.
- Probabilidad introductoria: media, varianza, distribuciones gaussianas.
- Python intermedio: NumPy, leer una funcion en `cv2`.

Si te faltan piezas, hay un apendice de **Matematica de bolsillo** al final con los resultados que vamos a usar.

Convenciones

- Una *imagen* se denota $I : \Omega \to \mathbb{R}^c$, con dominio $\Omega \subset \mathbb{Z}^2$ y c canales.
- El pixel en posicion (x, y) se escribe $I(x, y)$ o $I[y, x]$ (estilo NumPy: fila primero).
- Negrita para vectores y matrices ($\mathbf{x}$, $\mathbf{K}$); minuscula normal para escalares.
- Bloques azules = conexion con tu codigo. Bloques verdes = intuicion fisica. Bloques naranjas = ejercicios.

Índice general

Prefacio iii

I Fundamentos — la imagen como dato 1

1. La imagen digital 3
1.1. Por que importa . . . 3
1.2. Definicion formal . . . 3
1.2.1. Resolucion espacial vs resolucion radiometrica . . . 3
1.3. Espacios de color . . . 3
1.3.1. El error mas comun: BGR vs RGB . . . 4
1.4. Histograma . . . 4
1.4.1. Implementacion en NumPy puro . . . 4
1.5. Estadisticas globales . . . 5

2. Operaciones punto a punto 7
2.1. Por que importa . . . 7
2.2. Forma general . . . 7
2.2.1. Casos particulares . . . 7
2.3. Umbralizacion (thresholding) . . . 7
2.3.1. Otsu . . . 7
2.3.2. Triangle . . . 8
2.3.3. Adaptativo . . . 8
2.4. Ecualizacion de histograma . . . 8
2.4.1. Por que funciona . . . 8
2.4.2. CLAHE: ecualizacion adaptativa con limite de contraste . . . 8

3. Convoluciones y filtros lineales 11
3.1. Por que importa . . . 11
3.2. Definicion . . . 11
3.2.1. Operacion deslizante . . . 11
3.2.2. Padding y stride . . . 11
3.3. Propiedades fundamentales . . . 12
3.3.1. Linealidad . . . 12
3.3.2. Conmutatividad y asociatividad . . . 12
3.3.3. Separabilidad . . . 12
3.4. Catalogo de kernels . . . 12
3.4.1. Identidad . . . 12
3.4.2. Promedio (box filter) . . . 12
3.4.3. Gaussiano . . . 13
3.4.4. Sobel . . . 13
3.4.5. Laplaciano . . . 13

3.4.6. Sharpening (unsharp mask) . . . 13
3.5. Implementacion en NumPy . . . 14
3.5.1. Una implementacion vectorizada . . . 14
3.6. El teorema de la convolucion (anticipo) . . . 14

4. Filtros no lineales **17**
4.1. Por que importa . . . 17
4.2. Filtro mediana . . . 17
4.2.1. Por que funciona . . . 17
4.2.2. Costo . . . 17
4.3. Filtro bilateral . . . 17
4.4. Non-local means (NLM) . . . 18

5. Morfologia matematica **19**
5.1. Por que importa . . . 19
5.2. Morfologia binaria . . . 19
5.2.1. Apertura y cierre . . . 19
5.2.2. Propiedades . . . 20
5.3. Morfologia grayscale . . . 20
5.3.1. Top-hat y bottom-hat (black-hat) . . . 20
5.4. Elementos estructurantes . . . 20
5.5. Implementacion en NumPy . . . 21

II Estructura geometrica — bordes, regiones, formas **23**

6. Deteccion de bordes y el algoritmo de Canny **25**
6.1. Por que importa . . . 25
6.2. Modelo de borde . . . 25
6.3. Los cinco pasos de Canny . . . 25
6.3.1. Paso 1: Suavizado Gaussiano . . . 25
6.3.2. Paso 2: Magnitud y direccion del gradiente . . . 25
6.3.3. Paso 3: Supresion de no-maximos (NMS) . . . 25
6.3.4. Paso 4: Doble umbralizacion . . . 26
6.3.5. Paso 5: Histeresis . . . 26
6.4. Optimaidad de Canny . . . 26

7. Ridges y vesselness (Frangi-Hessian) **27**
7.1. Borde vs ridge: la distincion clave . . . 27
7.2. La matriz Hessiana . . . 27
7.3. Vesselness de Frangi . . . 27
7.3.1. Multi-escala . . . 28

8. Segmentacion clasica **29**
8.1. Definicion del problema . . . 29
8.2. Umbralizacion como segmentacion . . . 29
8.3. Region growing . . . 29
8.4. K-means y Mean Shift . . . 30
8.5. GrabCut . . . 30

9. Watershed: cuando los objetos se tocan **31**
9.1. La metafora . . . 31
9.2. Aplicacion a celulas tocandose . . . 31
9.3. Por que markers . . . 31
9.4. Algoritmo detallado . . . 31
9.4.1. Eligiendo τ (`min_distance`) . . . 32

10. Forma, momentos y descriptores **33**
10.1. Por que importa . . . 33
10.2. Momentos de imagen . . . 33
10.3. Descriptores de contorno . . . 33
10.3.1. Ajuste de elipse . . . 34
10.4. Cadenas de Freeman . . . 34

III El espacio de Fourier — la otra mitad del mundo **35**

11. Transformada de Fourier 2D **37**
11.1. Por que importa . . . 37
11.2. Definicion . . . 37
11.2.1. Magnitud y fase . . . 37
11.3. Propiedades fundamentales . . . 38
11.4. Filtrado en frecuencia . . . 38
11.4.1. Receta general . . . 38
11.4.2. Filtro paso bajo ideal (low-pass) . . . 38
11.4.3. Filtro paso alto . . . 38
11.4.4. Filtro paso banda . . . 38

12. Optica de Fourier para microscopia **41**
12.1. El microscopio como sistema lineal . . . 41
12.2. Apertura numerica y limite de difraccion . . . 41
12.3. Iluminacion oblicua = acceso a frecuencias altas . . . 41

13. Fourier Ptychographic Microscopy (FPM) **43**
13.1. El problema . . . 43
13.2. La idea de FPM . . . 43
13.3. Algoritmo de reconstruccion . . . 43
13.3.1. Cuantos angulos necesitas . . . 44

IV Multi-imagen — registracion y super-resolucion **45**

14. Registracion de imagenes **47**
14.0.1. Phase correlation . . . 47
14.0.2. ECC (Enhanced Correlation Coefficient) . . . 47
14.0.3. Feature-based: SIFT, ORB, AKAZE . . . 47

15. Super-resolucion **49**

V Aprendizaje profundo en vision 51

16. Del perceptron a la CNN 53
16.1. El perceptron . . . 53
16.2. La capa convolucional . . . 53
16.3. Pooling y receptive field . . . 53
16.4. Backpropagation . . . 53

17. U-Net y segmentacion semantica 55
17.1. La arquitectura U-Net . . . 55
17.2. Loss functions para segmentacion . . . 55

18. Modelos especificos para microscopia 57
18.1. Cellpose . . . 57
18.2. StarDist . . . 57
18.3. Real-ESRGAN . . . 57

19. Validacion y metricas 59
19.1. Para clasificacion . . . 59
19.2. Para segmentacion semantica . . . 59
19.3. Para segmentacion por instancias . . . 59
19.4. Por que no tener metricas es no tener pipeline . . . 59

VI Vanguardia 61

20. Self-supervised learning 63
20.1. La idea . . . 63
20.1.1. Noise2Void (N2V) . . . 63
20.1.2. Contrastive learning (SimCLR, MoCo, DINO) . . . 63

21. Foundation models 65
21.1. SAM — Segment Anything . . . 65
21.2. DINO / DINOv2 . . . 65
21.3. Modelos de difusion (Stable Diffusion, etc.) . . . 65

22. NeRF y 3D Gaussian Splatting 67
22.1. NeRF (Neural Radiance Fields) . . . 67
22.2. Gaussian Splatting . . . 67

Apendice A — Matematica de bolsillo 69
22.3. Algebra lineal . . . 69
22.4. Calculo . . . 69
22.5. Probabilidad . . . 69

Apendice B — Hoja de ruta de aprendizaje 71
22.6. Plan de 6 meses (10 horas/semana) . . . 71
22.7. Recursos imprescindibles . . . 71
22.8. Prediccion . . . 71

Bibliografia 73

Parte I

Fundamentos — la imagen como dato

Capítulo 1

La imagen digital

1.1. Por que importa

Antes de detectar una celula, restaurar una foto borrosa o entrenar una red, necesitas saber *con exactitud* que es lo que tu computadora ve. Cada decision posterior — el rango de un umbral, la varianza de un denoiser, el tipo de loss en una red — depende de esta primera capa.

1.2. Definicion formal

Definicion 1.1 (Imagen digital)**.** *Una* imagen digital *es una funcion* $I : \Omega \to V$ *donde el dominio* $\Omega \subset \mathbb{Z}^2$ *es una grilla rectangular finita* $\{0, \ldots, W-1\} \times \{0, \ldots, H-1\}$ *y el rango* V *es un subconjunto de* $\mathbb{R}^c$ *con* c *canales.*

En la practica $V = \{0, 1, \ldots, 2^b-1\}^c$ donde b es la profundidad de bits del canal. Las opciones tipicas:

dtype NumPy	Rango	Uso tipico
`uint8`	$[0, 255]$	JPEG, displays, foto consumer
`uint16`	$[0, 65535]$	camaras cientificas (microscopia, astronomia)
`float32`	$[-\infty, \infty]$	procesado interno, redes neuronales
`float64`	doble precision	calculos finos, no tipico en CV

1.2.1. Resolucion espacial vs resolucion radiometrica

Resolucion espacial: cuantos pixeles cubren un mismo angulo solido (foco, sensor). Se mide en *lineas por milimetro* (lp/mm) o en *ground sampling distance* (GSD) si tenes calibracion.

Resolucion radiometrica: cuantos niveles de gris distinguibles tenes por pixel (2^b). Una imagen `uint8` con 256 niveles puede ocultar detalles que una `uint16` con 65k niveles preserva.

Nota. *Tu sensor OV5647 da* `uint10` *crudos (1024 niveles) que el driver expone como* `uint16`*. Si despues de cargar la imagen haces* `img.astype(uint8)` *sin remapear estas tirando casi 100 niveles utiles a la basura. Por eso en* `preprocess.load_image` *ves* `cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)`*: estira el rango activo antes de bajar a 8 bits.*

1.3. Espacios de color

Una imagen color tiene $c = 3$ canales. Las representaciones que vas a encontrar:

Espacio	Canales	Cuando
RGB	rojo, verde, azul	estandar de displays. Matplotlib usa este orden
BGR	azul, verde, rojo	el orden por defecto en OpenCV. Si te equivocas, los azules se ven rojos
HSV	matiz, saturacion, valor	para segmentar por color (frutas, defectos visuales)
Lab	luminancia + dos cromaticos	uniforme perceptualmente. Bueno para diferencias de color
YUV / YCbCr	luma + croma	video, JPEG. Compresion eficiente del croma

La conversion entre RGB y grayscale tiene un coeficiente perceptual:

$$Y = 0{,}299R + 0{,}587G + 0{,}114B \tag{1.1}$$

Estos pesos vienen del estandar de television BT.601 y respetan que el ojo humano es mas sensible al verde.

1.3.1. El error mas comun: BGR vs RGB

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("celula.png")  # BGR
plt.imshow(img)                  # plt asume RGB --- los rojos se ven azules
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))  # correcto
```

Listing 1.1: Trampa clasica de OpenCV.

1.4. Histograma

Definicion 1.2 (Histograma)**.** *Para una imagen en escala de grises I con L niveles posibles, el* histograma *es la funcion $h : \{0, \ldots, L-1\} \to \mathbb{N}$ definida por*

$$h(k) = |\{(x, y) \in \Omega : I(x, y) = k\}|. \tag{1.2}$$

La distribucion de probabilidad empirica *se obtiene normalizando: $p(k) = h(k)/|\Omega|$.*

El histograma resume *cuanto* contraste tiene la imagen y *donde* esta concentrada la energia. Una imagen subexpuesta tiene su masa apilada cerca de cero; una saturada, cerca de $L-1$. El area total siempre es $|\Omega| = WH$.

1.4.1. Implementacion en NumPy puro

```
import numpy as np

def hist(img: np.ndarray, L: int = 256) -> np.ndarray:
    """Retorna h[k] = numero de pixeles con valor k."""
    h = np.zeros(L, dtype=np.int64)
    for k in range(L):
        h[k] = np.sum(img == k)
    return h

# Vectorizado (mas rapido):
```

```python
def hist_fast(img: np.ndarray, L: int = 256) -> np.ndarray:
    return np.bincount(img.ravel(), minlength=L)
```

Listing 1.2: Histograma desde cero.

Intuicion

El histograma es un *proyector*: pierde toda la informacion espacial. Dos imagenes muy diferentes pueden tener el mismo histograma si tienen los mismos pixeles, solo reordenados. Por eso operaciones que dependen *solo* del histograma (Capitulo 2) son intrinsicamente locales o globales en valor, nunca espaciales.

1.5. Estadisticas globales

Resumenes utiles para diagnosticar una imagen:

$$\text{media} \quad \mu = \frac{1}{|\Omega|} \sum_{(x,y)} I(x,y) \tag{1.3}$$

$$\text{varianza} \quad \sigma^2 = \frac{1}{|\Omega|} \sum_{(x,y)} (I(x,y) - \mu)^2 \tag{1.4}$$

$$\text{entropia} \quad H = -\sum_{k} p(k) \log_2 p(k) \tag{1.5}$$

La entropia mide *cuanta informacion* contiene el histograma: vale 0 si la imagen es uniforme y $\log_2 L$ si todos los niveles son igualmente probables. Es la cota superior de cuantos bits por pixel necesitas para almacenarla sin perdidas.

Conexion con el pipeline FPM-CubeSat

En `pipeline/preprocess.py`, la funcion `remove_vignette` usa la *mediana* (no la media) de los pixeles validos para sustituir el viñeteo. La mediana es *robusta a outliers*: una zona negra extrema no le afecta. Verificalo: aplicale `remove_vignette` a una imagen tuya y graficale el histograma antes/despues.

Ejercicios

1. Cargar `Imagenes/Variados/MicroscopiaPruebaCLAHE.png` con `cv2.imread`. Imprimir su `shape`, `dtype`, valor min, max, media, mediana.
2. Calcular el histograma con tu funcion `hist` y con `np.bincount`. Medir tiempos. Explicar.
3. Convertir a HSV con `cv2.cvtColor`, mostrar los tres canales por separado. ¿En cual se distingue mejor la celula del fondo? Explicar fisicamente por que.
4. Calcular la entropia. Aplicar `cv2.equalizeHist` y volver a calcular: ¿que cambio?

Capítulo 2

Operaciones punto a punto

2.1. Por que importa

Estas son las operaciones mas baratas del catalogo: cada pixel se transforma usando *solo su propio valor*, sin mirar a sus vecinos. Pese a la simplicidad, te permiten hacer la mitad del trabajo: realzar contraste, segmentar por umbral, corregir gamma, normalizar para una red.

2.2. Forma general

Definicion 2.1 (Operacion punto a punto)**.** *Es cualquier transformacion $I' = T(I)$ tal que existe una funcion $f : V \to V'$ con*

$$I'(x, y) = f(I(x, y)) \quad \forall (x, y) \in \Omega. \tag{2.1}$$

La funcion f se puede precalcular en una *look-up table* (LUT) de L entradas y aplicarse en $O(WH)$ con una sola indexacion vectorizada.

2.2.1. Casos particulares

Brillo aditivo: $f(v) = v + \beta$, con *clipping* a $[0, L-1]$.
Contraste multiplicativo: $f(v) = \alpha v$.
Brillo + contraste combinados: $f(v) = \alpha v + \beta$.
Inversion: $f(v) = (L-1) - v$. Util para invertir bright-field a algo parecido a fluorescencia (las paredes oscuras quedan brillantes).
Gamma: $f(v) = (L-1)\left(\frac{v}{L-1}\right)^{\gamma}$. Si $\gamma < 1$ aclara las sombras; si $\gamma > 1$ oscurece. Modela la respuesta no lineal de los displays CRT y compensa la sensibilidad logaritmica del ojo.

2.3. Umbralizacion (thresholding)

$$f_T(v) = \begin{cases} L-1 & \text{si } v \geq T \\ 0 & \text{si no} \end{cases} \tag{2.2}$$

El gran problema: ¿como elegir T? Tres respuestas clasicas:

2.3.1. Otsu

Otsu busca el T que minimiza la *varianza intra-clase*, equivalente a maximizar la varianza entre clases:

$$T^* = \arg\max_T \ \omega_0(T)\omega_1(T)(\mu_0(T) - \mu_1(T))^2, \tag{2.3}$$

donde ω_i es la masa de la clase i y μ_i su media. Funciona bien cuando el histograma es *bimodal*.

2.3.2. Triangle

Si el histograma tiene un solo pico (microscopia con poco contraste suele ser asi), Otsu falla. El metodo del triangulo conecta el pico con el extremo opuesto y elige el T donde la perpendicular a esa linea es maxima.

2.3.3. Adaptativo

$$T(x, y) = \mu_W(x, y) - C \tag{2.4}$$

donde μ_W es la media local en una ventana W alrededor de (x, y) y C un offset. Util cuando la iluminacion no es uniforme.

```
def otsu(img: np.ndarray) -> int:
    h = np.bincount(img.ravel(), minlength=256).astype(np.float64)
    p = h / h.sum()
    omega = np.cumsum(p)                # prob acumulada
    mu = np.cumsum(p * np.arange(256))
    mu_total = mu[-1]
    # varianza entre clases para cada T candidato
    sigma_b2 = (mu_total * omega - mu)**2 / (omega * (1 - omega) + 1e-12)
    return int(np.argmax(sigma_b2))
```

Listing 2.1: Otsu en NumPy puro.

2.4. Ecualizacion de histograma

Intuicion

Si una imagen tiene su masa concentrada en un rango estrecho (digamos $[80, 130]$), el contraste percibido es bajo. *Ecualizar* significa redistribuir esa masa para que cada nivel de gris tenga aproximadamente la misma probabilidad: el histograma queda lo mas plano posible. Es como aplicar una funcion de transferencia que estira las zonas densas y comprime las vacias.

La transformacion de ecualizacion es exactamente la *funcion de distribucion acumulada* (CDF) reescalada:

$$T(k) = (L - 1) \cdot F(k) = (L - 1) \sum_{j=0}^{k} p(j), \tag{2.5}$$

donde F es la CDF empirica.

2.4.1. Por que funciona

Teorema 2.2. *Sea X una variable aleatoria con CDF F_X continua. Entonces $Y = F_X(X)$ es uniforme en $[0, 1]$.*

Es decir: aplicar la CDF como funcion de transferencia uniformiza la distribucion. La version digital es discreta, asi que la igualdad es aproximada.

2.4.2. CLAHE: ecualizacion adaptativa con limite de contraste

La ecualizacion global tiene dos defectos:

1. Si una zona pequeña tiene un detalle de bajo contraste, la CDF global lo borra (los pixeles relevantes son pocos comparados con el resto).

2. Si una zona tiene poco rango de valores, la ecualizacion amplifica ruido al estirar.

CLAHE (*Contrast Limited Adaptive Histogram Equalization*) resuelve ambos:

1. Divide la imagen en *tiles* (ej. 8×8).
2. Calcula el histograma local de cada tile.
3. *Recorta* el histograma a un `clipLimit`: cada bin que excede el limite se redistribuye uniformemente entre todos. Eso evita amplificar ruido.
4. Ecualiza cada tile con su histograma recortado.
5. Interpola bilinealmente entre tiles para evitar bordes visibles.

Conexion con el pipeline FPM-CubeSat

`config.yaml` controla CLAHE en la seccion `preprocess: clahe_clip_limit: 3.0` y `clahe_grid_size: 8`. La funcion en `pipeline/preprocess.py:apply_clahe` es un wrapper de tres lineas sobre `cv2.createCLAHE`. Tu tuner interactivo `tools/tune_onion.py` te deja barrer el clip limit en vivo.

Ejercicios

1. Implementar Otsu desde cero (codigo arriba) y comparar con `cv2.threshold(...cv2.THRESH_OTSU)`.
2. Implementar ecualizacion de histograma con (2.5) y comparar con `cv2.equalizeHist`.
3. Tomar una imagen con iluminacion no uniforme (ej. tu `Variados/MicroscopiaPruebaCLAHE.png`). Aplicar ecualizacion global, CLAHE con clip $= 2$, CLAHE con clip $= 10$, CLAHE con tile 4×4 vs 32×32. Reportar el efecto sobre la deteccion de paredes celulares con tu tuner.
4. Aplicar gamma $\gamma = 0{,}5$ y $\gamma = 2{,}0$. Verificar que la ecualizacion es invariante a transformaciones monotonicas crecientes (la CDF las absorbe).

Capítulo 3

Convoluciones y filtros lineales

3.1. Por que importa

La convolucion es *el* verbo de la vision por computadora. Casi todo — suavizar, afilar, detectar bordes, las redes neuronales — es una convolucion bajo el capot. Si entendes este capitulo, entendes el 60 % del area.

3.2. Definicion

Definicion 3.1 (Convolucion 2D)**.** *Sean* $I : \mathbb{Z}^2 \to \mathbb{R}$ *una imagen y* $K : \mathbb{Z}^2 \to \mathbb{R}$ *un* kernel *de soporte finito* $\{-r, \ldots, r\}^2$*. La* convolucion *de* I *con* K *es*

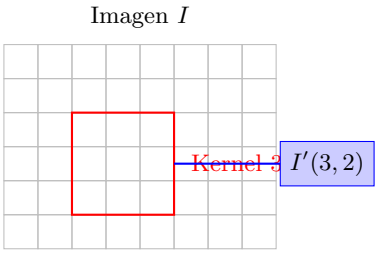
$$(I * K)(x, y) = \sum_{i=-r}^{r} \sum_{j=-r}^{r} I(x - i, y - j) K(i, j). \tag{3.1}$$

La diferencia con la *correlacion* esta en el signo de los indices:

$$(I \star K)(x, y) = \sum_{i,j} I(x + i, y + j) K(i, j). \tag{3.2}$$

La correlacion no *voltea* el kernel; la convolucion si. Para kernels simetricos (como los Gaussianos) son lo mismo. `cv2.filter2D` implementa correlacion; los frameworks de deep learning tambien (a pesar del nombre).

3.2.1. Operacion deslizante



El kernel se desliza por toda la imagen; en cada posicion calcula un *producto interno* con la region cubierta y deposita el resultado en la salida.

3.2.2. Padding y stride

¿Que pasa en los bordes? Tres opciones tipicas:

- `valid`: salida mas chica que la entrada (no calcula donde el kernel se sale).
- `same` con `zero`: rellena el exterior con ceros. Simple pero introduce ringing.
- `same` con `reflect`: refleja la imagen sobre el borde. Casi siempre la mejor opcion.

El *stride* s es el paso entre posiciones consecutivas. $s = 1$ produce salida del mismo tamaño (con padding); $s > 1$ submuestrea. Las redes convolucionales usan stride para reducir resolucion sin pooling explicito.

3.3. Propiedades fundamentales

3.3.1. Linealidad

$$(\alpha I_1 + \beta I_2) * K = \alpha(I_1 * K) + \beta(I_2 * K). \tag{3.3}$$

Permite descomponer una imagen ruidosa $I = S + N$ y notar que el filtrado actua sobre señal y ruido por separado.

3.3.2. Conmutatividad y asociatividad

$$I * K = K * I \tag{3.4}$$

$$(I * K_1) * K_2 = I * (K_1 * K_2) \tag{3.5}$$

La asociatividad es magica: aplicar dos filtros consecutivos equivale a aplicar uno solo (su *convolucion*). En particular, dos Gaussianos sucesivos $G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_2^2}}$.

3.3.3. Separabilidad

Un kernel K es *separable* si existe K_x, K_y unidimensionales tales que $K(i,j) = K_x(i)K_y(j)$. Entonces:

$$I * K = (I * K_y) * K_x \tag{3.6}$$

y se hace primero el filtrado vertical (1D), despues el horizontal (1D). Costo: $O(WH \cdot 2r)$ en lugar de $O(WH \cdot r^2)$. Para un kernel 11×11 es $11/121 \approx 9\,\%$ del costo original.

El Gaussiano y los Sobel son separables. El Laplaciano *no* lo es.

3.4. Catalogo de kernels

3.4.1. Identidad

$$K_{\text{id}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \tag{3.7}$$

Devuelve la imagen sin cambios. Util para depurar tu implementacion.

3.4.2. Promedio (box filter)

$$K_{\text{box}} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \tag{3.8}$$

Suaviza promediando vecinos. Genera artefactos cuadrados.

3.4.3. Gaussiano

$$G_\sigma(i,j) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{i^2+j^2}{2\sigma^2}\right). \tag{3.9}$$

Es *el* filtro de suavizado de la disciplina. Propiedades:

- Separable.
- Optimo en el sentido de minimizar incertidumbre espacio-frecuencia (Heisenberg).
- En escala logaritmica de σ define un *scale-space* que es invariante a traslaciones y rotaciones.
- Tamaño de kernel sugerido: $k = 2\lceil 3\sigma \rceil + 1$ (atrapa el 99.7 % de la masa).

3.4.4. Sobel

Aproximaciones discretas a las derivadas parciales:

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \tag{3.10}$$

El gradiente discreto se aproxima como

$$\nabla I(x,y) \approx \begin{pmatrix} (I * S_x)(x,y) \\ (I * S_y)(x,y) \end{pmatrix}. \tag{3.11}$$

La magnitud $|\nabla I| = \sqrt{(I * S_x)^2 + (I * S_y)^2}$ es el primer paso de Canny (Cap. 6).

3.4.5. Laplaciano

$$L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad \text{aproxima} \quad \nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}. \tag{3.12}$$

El cero del Laplaciano corresponde a un *punto de inflexion*: la base del detector de bordes Marr-Hildreth (LoG — Laplacian of Gaussian) y de blob detectors (DoG — Difference of Gaussians).

3.4.6. Sharpening (unsharp mask)

$$I_{\text{sharp}} = I + \lambda(I - I * G_\sigma) = I * (\delta + \lambda(\delta - G_\sigma)) \tag{3.13}$$

Se hace borrosa una copia, se resta de la original (queda solo lo "agudo": detalles), se suma con un peso. Equivalente a convolucionar con un kernel "centro alto, anillo negativo".

3.5. Implementacion en NumPy

```
import numpy as np

def conv2d_naive(img: np.ndarray, k: np.ndarray) -> np.ndarray:
    """Convolucion 2D, valid mode, sin padding. O(W H r^2)."""
    H, W = img.shape
    kr = k.shape[0] // 2
    out = np.zeros((H - 2*kr, W - 2*kr), dtype=np.float64)
    k_flip = k[::-1, ::-1]  # convolucion = correlacion(kernel volteado)
    for y in range(kr, H - kr):
        for x in range(kr, W - kr):
            patch = img[y-kr:y+kr+1, x-kr:x+kr+1]
            out[y-kr, x-kr] = np.sum(patch * k_flip)
    return out
```

Listing 3.1: Convolucion 2D con doble for (la version educativa).

Esta version corre en O(W H r^2) operaciones *en Python*: para una imagen 1000×1000 con kernel 5×5 son 25 millones de iteraciones, cada una con overhead del interprete. Vas a ver `60-90 s`. La version OpenCV con `cv2.filter2D` demora $\sim$ 30 ms.

¿Por que la diferencia? Tres razones:

1. OpenCV esta escrito en C++.
2. Usa SIMD (instrucciones SSE/AVX que procesan 8 floats por ciclo).
3. Si detecta separabilidad, la aplica.

3.5.1. Una implementacion vectorizada

Si convertis la convolucion a una multiplicacion de matrices (*im2col*), NumPy te da los SIMD gratis:

```
def im2col(img: np.ndarray, kh: int, kw: int) -> np.ndarray:
    H, W = img.shape
    out_h, out_w = H - kh + 1, W - kw + 1
    cols = np.zeros((kh*kw, out_h*out_w), dtype=img.dtype)
    idx = 0
    for i in range(kh):
        for j in range(kw):
            cols[idx] = img[i:i+out_h, j:j+out_w].ravel()
            idx += 1
    return cols, out_h, out_w

def conv2d_vec(img, k):
    cols, H, W = im2col(img, *k.shape)
    out = (k[::-1, ::-1].ravel() @ cols).reshape(H, W)
    return out
```

Listing 3.2: Convolucion via im2col (NumPy vectorizado).

Esto baja a 0.5–1 s — todavia 30x mas lento que OpenCV pero mil veces mas rapido que el ciclo doble.

3.6. El teorema de la convolucion (anticipo)

Teorema 3.2 (Teorema de la convolucion)**.** *Sean I, K funciones suficientemente regulares y $\mathcal{F}$ la transformada de Fourier 2D. Entonces:*

$$\mathcal{F}\{I * K\} = \mathcal{F}\{I\} \cdot \mathcal{F}\{K\}. \tag{3.14}$$

Es decir: convolucionar en espacio = multiplicar en frecuencia. Para kernels grandes ($r > 15$), pasar por FFT es mucho mas rapido. Vamos a desarrollar esto en la Parte III.

Conexion con el pipeline FPM-CubeSat

`cv2.GaussianBlur` en `pipeline/segmentation_onion.py:segment_opencv` convoluciona con un kernel Gaussiano separable. `cv2.Canny` hace internamente Sobel (los kernels de la seccion 3.4.4). Tu `tune_onion.py` usa Frangi, que es un detector basado en *segundas* derivadas (matriz Hessiana — ver Cap. 7).

Ejercicios

1. Implementar `conv2d_naive` y `conv2d_vec`. Aplicar Sobel x e y a una imagen tuya. Sumar las magnitudes y mostrar.
2. Comparar tiempos con `cv2.filter2D`. Hacer una tabla con kernels 3×3, 7×7, 15×15, 31×31.
3. Generar un Gaussiano discreto con (3.9) para $\sigma = 1, 2, 5$. Convolucionar y mostrar. Visualizar el kernel como `plt.imshow`.
4. Verificar la propiedad de cascada: $G_1 * G_2 \stackrel{?}{=} G_{\sqrt{2}}$ (convolver dos Gaussianos $\sigma = 1$ y comparar con un solo Gaussiano $\sigma = \sqrt{2}$).
5. Implementar *unsharp mask*. Probar con $\lambda = 0{,}5, 1{,}0, 2{,}0, 5{,}0$. ¿A partir de cuanto se ve ruido amplificado?

Capítulo 4

Filtros no lineales

4.1. Por que importa

Los filtros lineales tienen un compromiso brutal: si suavizas para sacar ruido, suavizas tambien los bordes. Los filtros no lineales rompen ese compromiso usando informacion local: solo promedian pixeles *parecidos*.

4.2. Filtro mediana

$$I'(x, y) = \text{med}\{I(x', y') : (x', y') \in W(x, y)\} \tag{4.1}$$

donde $W(x, y)$ es la ventana centrada en (x, y).

4.2.1. Por que funciona

La mediana es *robusta*: cambia poco si reemplazas hasta el 50 % de los datos por valores arbitrarios (*breakdown point* = 0.5). Por eso es perfecto para ruido *sal y pimienta*: pixeles aislados extremadamente claros u oscuros.

4.2.2. Costo

Una implementacion ingenua ordena los r^2 vecinos por cada pixel: $O(WH \cdot r^2 \log r)$. Con histogramas locales actualizados se baja a $O(WH \cdot r)$. OpenCV usa la version optimizada.

4.3. Filtro bilateral

El filtro bilateral pesa los vecinos por dos criterios: cercania espacial y similitud en valor.

$$I'(x, y) = \frac{1}{Z(x, y)} \sum_{(x', y')} I(x', y') \cdot G_{\sigma_s}(\|p - p'\|) \cdot G_{\sigma_r}(|I(x, y) - I(x', y')|), \tag{4.2}$$

con Z el factor de normalizacion. Resultado: dentro de una region homogenea suaviza como un Gaussiano; cerca de un borde, los vecinos del otro lado tienen peso de *intensidad* bajo y casi no participan. Por eso *preserva bordes*.

Parametros:

- σ_s (`sigmaSpace`): radio espacial. Tipicamente 5–15 pixeles.
- σ_r (`sigmaColor`): tolerancia en intensidad. Tipicamente 10–40 (en uint8).

4.4. Non-local means (NLM)

Intuicion

La idea: si una imagen tiene texturas repetidas, no necesitas mirar solo los *vecinos espaciales* para promediar. Mira todos los *parches similares* en cualquier parte de la imagen. Es exactamente lo que vos haces cuando intuis: "este pixel deberia ser X porque otros parches parecidos en la imagen tienen X". Funciona ridiculamente bien para microscopia, donde texturas se repiten.

$$I'(x,y) = \frac{1}{Z(x,y)} \sum_{(x',y')\in S} I(x',y') \exp\left(-\frac{\|P(x,y) - P(x',y')\|^2}{h^2}\right), \tag{4.3}$$

donde $P(\cdot)$ es un *parche* alrededor del pixel y h controla la fuerza del filtro.

Costo: pesado, $O(WH \cdot |S| \cdot p^2)$ con $|S|$ tamaño de busqueda y p tamaño del parche. La version de OpenCV (`cv2.fastNlMeansDenoising`) usa varios trucos (integral images, busqueda restringida).

Conexion con el pipeline FPM-CubeSat

`pipeline/preprocess.py:denoise` llama a `cv2.fastNlMeansDenoising`. El parametro `denoise_strength` en `config.yaml` es el h de la formula. Valores tipicos para microscopia: 5–15. Mas alto = mas suavizado, riesgo de borrar paredes finas.

Ejercicios

1. Implementar mediana 3×3 con `np.median` sobre una ventana deslizante. Comparar con `cv2.medianBlur`.
2. Tomar una imagen tuya y agregarle ruido sal-pimienta artificial: `img[np.random.rand(*img.shape) < 0.05] = 0; img[np.random.rand(*img.shape) < 0.05] = 255`. Comparar Gaussian blur vs mediana 3×3.
3. Aplicar `cv2.bilateralFilter` con $(\sigma_s, \sigma_r) = (10,10), (10,50), (10,200)$. Comparar el efecto sobre las paredes de cebolla.
4. Comparar tiempo y calidad: NLM vs bilateral vs Gaussian. Reportar tabla.

Capítulo 5

Morfologia matematica

5.1. Por que importa

La morfologia es el algebra de las formas binarias y grayscale. Te permite limpiar mascaras (sacar piquitos, llenar agujeros), realzar features finos (las paredes de tu cebolla), y construir piezas para cualquier segmentacion. Es la herramienta mas *barata* y mas *robusta* del catalogo clasico.

5.2. Morfologia binaria

Sea $A \subset \mathbb{Z}^2$ el conjunto de pixeles "blancos" de una imagen binaria (objeto), y $B \subset \mathbb{Z}^2$ el *elemento estructurante* (kernel binario, generalmente con forma de cuadrado, cruz o circulo).

Definicion 5.1 (Erosion)**.**

$$A \ominus B = \{z \in \mathbb{Z}^2 : B_z \subseteq A\} \tag{5.1}$$

donde $B_z = \{b + z : b \in B\}$ *es* B *trasladado a* z*.*

Intuicion: el centro del kernel "permanece" en A solo si *toda* la forma del kernel cabe dentro de A. La erosion *encoge* el objeto y elimina detalles mas pequeños que B.

Definicion 5.2 (Dilatacion)**.**

$$A \oplus B = \bigcup_{a \in A} B_a = \{z : B_z \cap A \neq \emptyset\} \tag{5.2}$$

Intuicion: la dilatacion *agranda* el objeto: cada pixel del objeto pinta una copia del kernel.

5.2.1. Apertura y cierre

$$A \circ B = (A \ominus B) \oplus B \quad \text{(apertura)} \tag{5.3}$$

$$A \bullet B = (A \oplus B) \ominus B \quad \text{(cierre)} \tag{5.4}$$

Apertura: erosion seguida de dilatacion. Saca features mas chicos que B (*ruido aislado, salientes finos*) y suaviza el contorno.

Cierre: dilatacion seguida de erosion. Cierra agujeros mas chicos que B y conecta partes cercanas.

5.2.2. Propiedades

- **Idempotencia**: $A \circ B \circ B = A \circ B$ y $A \bullet B \bullet B = A \bullet B$. Aplicar apertura/cierre dos veces es lo mismo que una.
- **Dualidad**: $A \ominus B = (A^c \oplus \tilde{B})^c$ donde $\tilde{B}$ es el reflejo de B por el origen. Erosion y dilatacion son duales bajo complemento.
- **Crecimiento monotono**: $A \subseteq A'$ implica $A \ominus B \subseteq A' \ominus B$.

5.3. Morfologia grayscale

Generalizamos a imagenes en escala de grises:

$$(I \ominus B)(x, y) = \min_{(i,j)\in B} I(x + i, y + j) \tag{5.5}$$

$$(I \oplus B)(x, y) = \max_{(i,j)\in B} I(x + i, y + j) \tag{5.6}$$

Erosion grayscale = minimo local: oscurece detalles claros mas chicos que B.
Dilatacion grayscale = maximo local: aclara detalles oscuros mas chicos que B.

5.3.1. Top-hat y bottom-hat (black-hat)

$$\text{Tophat}(I) = I - (I \circ B) \quad \text{(realza features claros chicos)} \tag{5.7}$$

$$\text{Blackhat}(I) = (I \bullet B) - I \quad \text{(realza features oscuros chicos)} \tag{5.8}$$

Tophat: dado que $I \circ B$ remueve features claros mas chicos que B, restarlo aisla esos features.
Blackhat: simetricamente, aisla los features *oscuros*.

Intuicion

Las paredes de tu cebolla son lineas oscuras finas sobre fondo claro. Un `blackhat` con un disco de radio mayor que el grosor de pared las aisla casi perfectamente. Por eso `tools/tune_onion.py` las usa antes del Frangi.

5.4. Elementos estructurantes

Forma OpenCV	Codigo	Cuando usar
Rectangulo	`MORPH_RECT`	Default. Rapido.
Elipse / disco	`MORPH_ELLIPSE`	Cuando los objetos son aproximadamente circulares (microesferas, nucleos).
Cruz	`MORPH_CROSS`	Cuando hay estructuras alineadas a ejes.

El *tamaño* importa: erodar con un kernel mas grande que tus features los borra completamente. Heuristica: para limpiar ruido aislado, $k \approx 3$. Para sacar zonas amplias, $k \approx$ tamaño tipico del feature.

5.5. Implementacion en NumPy

```python
def erode_gray(img: np.ndarray, B: np.ndarray) -> np.ndarray:
    """B es un array binario indicando el soporte del kernel."""
    H, W = img.shape
    kh, kw = B.shape
    rh, rw = kh // 2, kw // 2
    out = np.zeros_like(img)
    padded = np.pad(img, ((rh, rh), (rw, rw)), mode='edge')
    for y in range(H):
        for x in range(W):
            patch = padded[y:y+kh, x:x+kw]
            out[y, x] = np.min(patch[B > 0])
    return out
```

Listing 5.1: Erosion grayscale desde cero.

Conexion con el pipeline FPM-CubeSat

En `pipeline/segmentation_onion.py:segment_opencv` se aplica un `cv2.MORPH_CLOSE` a los bordes Canny para reconectar paredes rotas. En `tools/tune_onion.py:detect_walls` se usa un `MORPH_BLACKHAT` con disco elíptico para realzar paredes oscuras antes del Frangi. Si subis el `Tophat ksize` en el slider, estas cambiando el tamaño del kernel B.

Ejercicios

1. Implementar erode/dilate grayscale en NumPy. Comparar con `cv2.erode`/`cv2.dilate`.
2. Aplicar tophat y blackhat con disco $k = 15, 31, 61$ a una imagen tuya. ¿A partir de que tamaño dejas de captar paredes?
3. Verificar idempotencia: `cv2.morphologyEx(img, cv2.MORPH_OPEN, k)` aplicado dos veces = aplicado una vez.
4. Construir un `skeleton` (esqueleto de un objeto binario) iterativamente: $S = S \cup \{A^k - (A^k \circ B)\}$ donde A^k es la k-esima erosion.

Parte II

Estructura geometrica — bordes, regiones, formas

Capítulo 6

Deteccion de bordes y el algoritmo de Canny

6.1. Por que importa

Un *borde* es un cambio rapido en intensidad: la frontera entre dos regiones. Detectarlos es el paso obligado antes de muchas tareas (segmentacion, deteccion de objetos, analisis de forma) y un test de fuego sobre la teoria de derivadas en imagenes.

6.2. Modelo de borde

Modelamos un borde 1D ideal como una funcion escalon:

$$e(x) = \begin{cases} 0 & x < 0 \\ A & x \geq 0 \end{cases} \tag{6.1}$$

Su derivada es $A\delta(x)$ (un impulso en $x = 0$). La derivada *detecta* la posicion del borde y la *magnitud* estima el contraste.

En 2D, un borde es una curva sobre la cual la derivada direccional perpendicular es alta. La direccion en cada punto es la del *gradiente*:

$$\nabla I = (\partial_x I, \partial_y I), \qquad |\nabla I| = \sqrt{(\partial_x I)^2 + (\partial_y I)^2}. \tag{6.2}$$

6.3. Los cinco pasos de Canny

6.3.1. Paso 1: Suavizado Gaussiano

Aplicar G_σ para reducir ruido. Sin esto, el gradiente reacciona a cada bit aleatorio. Tipicamente $\sigma = 1$–2 pixeles.

6.3.2. Paso 2: Magnitud y direccion del gradiente

Convolver con S_x y S_y, calcular:

$$M(x, y) = \sqrt{(I * S_x)^2 + (I * S_y)^2}, \qquad \theta(x, y) = \arctan \frac{I * S_y}{I * S_x}. \tag{6.3}$$

6.3.3. Paso 3: Supresion de no-maximos (NMS)

M es ancha (varios pixeles). Para borde de un pixel: en cada (x, y), comparar $M(x, y)$ con sus dos vecinos en la *direccion del gradiente* θ. Si no es maximo, ponerlo en cero. Resultado: bordes finos.

6.3.4. Paso 4: Doble umbralizacion

Definir dos umbrales $T_{\text{lo}} < T_{\text{hi}}$:

- Pixeles con $M > T_{\text{hi}}$ son *strong*: ciertamente borde.
- Pixeles con $T_{\text{lo}} < M < T_{\text{hi}}$ son *weak*: candidatos.
- Pixeles con $M < T_{\text{lo}}$ se descartan.

6.3.5. Paso 5: Histeresis

Un *weak* se conserva solo si esta conectado por una cadena 8-conexa a algun *strong*. Implementacion: BFS/DFS desde cada strong propagando.

Hyper-parametros tipicos: $T_{\text{hi}} \in [100, 200]$ y $T_{\text{lo}} = T_{\text{hi}}/2$ o $T_{\text{hi}}/3$.

6.4. Optimaidad de Canny

Canny derivo el algoritmo optimizando tres criterios simultaneamente:

1. **Buena deteccion**: max probabilidad de marcar pixeles que son borde, min de marcar los que no.
2. **Buena localizacion**: los puntos detectados estan lo mas cerca posible del borde real.
3. **Respuesta unica**: un solo pixel marcado por borde real.

El detector optimo, bajo el modelo de borde escalon + ruido gaussiano blanco, es aproximadamente la primera derivada de un Gaussiano. Eso fundamenta los pasos 1+2.

Conexion con el pipeline FPM-CubeSat

`pipeline/segmentation_onion.py:segment_opencv` llama a `cv2.Canny(blurred, 30, 100)`. Los umbrales bajos (30, 100 sobre 255) atrapan paredes de bajo contraste pero introducen ruido. El blur previo es `GaussianBlur(...blur_kernel...)`.

Ejercicios

1. Implementar los 5 pasos de Canny en NumPy. Comparar con `cv2.Canny`. ¿Donde difieren?
2. Sobre tu imagen de cebolla, barrer $(T_{\text{lo}}, T_{\text{hi}}) \in \{(20, 80), (50, 150), (100, 200)\}$. ¿Cual atrapa mejor las paredes finas?
3. Cambiar el orden: hacer Canny *antes* de CLAHE. Reportar diferencia.
4. Implementar Marr-Hildreth (LoG): convolver con Gaussiano y calcular zero-crossings del Laplaciano. Comparar con Canny.

Capítulo 7

Ridges y vesselness (Frangi-Hessian)

7.1. Borde vs ridge: la distincion clave

Una pared de cebolla *no es un borde*: es una linea oscura fina rodeada de fondo claro. Un detector de bordes responde a las dos transiciones (fondo→pared y pared→fondo) como dos lineas separadas. Lo que necesitas es un detector de *ridges*: estructuras tubulares 1D embebidas en 2D.

7.2. La matriz Hessiana

Definicion 7.1 (Hessiana)**.** *Para una imagen suave I, la* matriz Hessiana *en (x, y) es*

$$H(x, y) = \begin{pmatrix} \partial_{xx} I & \partial_{xy} I \\ \partial_{xy} I & \partial_{yy} I \end{pmatrix}. \tag{7.1}$$

Sus dos autovalores $|\lambda_1| \leq |\lambda_2|$ y autovectores asociados describen la curvatura local:

- En una zona homogenea: $\lambda_1 \approx \lambda_2 \approx 0$.
- En un *blob* oscuro: ambos λ son grandes y positivos.
- En un *ridge oscuro*: $\lambda_1 \approx 0$ (bajo a lo largo del ridge), $\lambda_2 \gg 0$ (alto perpendicular a el).
- En un *ridge claro*: similar pero con λ negativos.

7.3. Vesselness de Frangi

Frangi (1998) propuso combinar los dos autovalores en un score:

$$\mathcal{V}(s) = \begin{cases} 0 & \text{si } \lambda_2 > 0 \text{ (para black ridges)} \\ \exp\left(-\dfrac{R_B^2}{2\beta^2}\right)\left(1 - \exp\left(-\dfrac{S^2}{2c^2}\right)\right) & \text{si no} \end{cases} \tag{7.2}$$

donde:

- $R_B = \lambda_1/\lambda_2$ es la *razon de excentricidad*: 0 en un ridge perfecto, 1 en un blob.
- $S = \sqrt{\lambda_1^2 + \lambda_2^2}$ es la *magnitud de estructura*: 0 en zona plana.
- β controla la sensibilidad a la forma (default 0.5).
- c se ajusta a la mitad de la S maxima en la imagen.

7.3.1. Multi-escala

La hessiana se calcula sobre la imagen suavizada con G_σ. Como una pared puede tener distintos grosores, se evalua para varios $\sigma \in \{\sigma_1, \ldots, \sigma_n\}$ y se toma el maximo:

$$\mathcal{V}^*(x, y) = \max_\sigma \mathcal{V}_\sigma(x, y). \tag{7.3}$$

Eso da invariancia a escala dentro de un rango.

Conexion con el pipeline FPM-CubeSat

`tools/tune_onion.py:detect_walls` usa `skimage.filters.frangi(..., black_ridges=True)`. El slider `Frangi smax` controla el sigma maximo del rango multi-escala. Si tus paredes son finas ($\sim$ 2px), $\sigma_{\text{máx}} = 4$ alcanza; si son gruesas ($\sim$ 8px), subir a $\sigma_{\text{máx}} = 12$.

Ejercicios

1. Calcular la Hessiana de una imagen I usando convoluciones con kernels de segunda derivada: $\partial_{xx} = [1, -2, 1]$ etc.
2. Calcular sus autovalores en cada pixel (usar `np.linalg.eigvalsh` o las formulas cerradas para matrices 2×2).
3. Implementar el score de Frangi y compararlo con `skimage.filters.frangi`.
4. Probar Sato (alternativa popular): $\mathcal{V}_{\text{Sato}} = -\lambda_1$ si $\lambda_1 < 0$ y $\lambda_2 < 0$, sino 0.

Capítulo 8

Segmentacion clasica

8.1. Definicion del problema

Definicion 8.1 (Segmentacion)**.** *Dada una imagen $I : \Omega \to V$, una* segmentacion *es una particion $\Omega = \bigcup_k R_k$ tal que cada R_k es conexa y los pixeles dentro de R_k comparten alguna propiedad relevante (color, textura, semantica).*

Hay dos sabores principales:

- **Semantica**: cada pixel recibe una clase (*celula*, *fondo*). No distingue instancias.
- **Por instancias**: cada pixel recibe ademas un *identificador* (*celula 1*, *celula 2*). Es lo que necesitas para contar.

8.2. Umbralizacion como segmentacion

El caso mas simple: dos clases (objeto vs fondo) determinadas por un umbral T. Vimos Otsu en el Cap. 2. Limitaciones:

- No usa informacion espacial: dos pixeles aislados con el mismo nivel quedan en la misma clase.
- No distingue instancias.
- Falla con iluminacion no uniforme (de ahi el adaptativo).

8.3. Region growing

Algoritmo:

1. Sembrar un seed s_0.
2. Inicializar la region $R = \{s_0\}$.
3. Mientras haya vecinos p de R que cumplan un criterio (ej. $|I(p) - \mu_R| < \tau$): agregarlos.
4. Repetir.

Sensibilidad a seeds y al criterio. La version moderna (Cap. 9) usa watershed en lugar de region growing puro.

8.4. K-means y Mean Shift

K-means agrupa pixeles en k clusters minimizando la varianza intra-cluster. Trata la imagen como un conjunto de vectores en $\mathbb{R}^c$ ignorando posicion espacial. Util como preprocesamiento.

Mean shift es no parametrico (no necesita k): cada pixel "asciende" al modo mas cercano de la densidad estimada por kernel. Da segmentacion natural pero costoso.

8.5. GrabCut

Combina modelos de mezcla gaussiana (GMM) sobre fondo y objeto con un grafo de cortes minimos. Lo usas con un rectangulo aproximado: el algoritmo refina hacia el contorno preciso.

Conexion con el pipeline FPM-CubeSat

La via `segment_opencv` de tu pipeline es *semantica* (binaria: pared vs interior) seguida de *etiquetado por contornos*. No usa region growing ni watershed — por eso fusiona celulas tocandose. La mejora del Cap. 9 arregla justo eso.

Capítulo 9

Watershed: cuando los objetos se tocan

9.1. La metafora

Pensa la imagen en escala de grises como un paisaje topografico: pixeles oscuros son *valles*, claros son *cumbres*. Imagina que llueve uniformemente y el agua *inunda* los valles. Cuando dos cuencas estan a punto de fusionarse, construis una *represa*. Las represas finales son los *watershed lines*: separan cuencas naturales.

9.2. Aplicacion a celulas tocandose

Para una imagen donde quieres separar celulas tocandose:

1. Detectar la mascara binaria del foreground (pixeles de celula).
2. Calcular la *transformada de distancia* $D(x, y)$ = distancia al pixel de fondo mas cercano. Los *maximos locales* de D son los centros de las celulas.
3. Esos maximos son tus *markers* (semillas).
4. Aplicar watershed sobre $-D$ con esos markers: cada marker se expande hasta encontrar otra cuenca o el fondo.

9.3. Por que markers

Sin markers, watershed sobre el gradiente sufre de *sobre-segmentacion*: cualquier minimo local define una cuenca. La version *controlada por marcadores* (Beucher, 1990) elimina ese problema: las cuencas solo se generan donde haya un marker.

9.4. Algoritmo detallado

1. Calcular D = distancia al fondo (`cv2.distanceTransform`).
2. Encontrar maximos locales: $\text{seed}(x, y) = 1 \iff D(x, y) > D(x', y')$ para todos los (x', y') en una vecindad de radio τ (`skimage.feature.peak_local_max`).
3. Asignar a cada seed un id distinto $\rightarrow$ *markers*.
4. Aplicar `skimage.segmentation.watershed(-D, markers, mask=foreground)`.

9.4.1. Eligiendo τ (`min_distance`)

- τ pequeño: muchos seeds $\rightarrow$ sobre-segmentacion.
- τ grande: pocos seeds $\rightarrow$ varias celulas se fusionan.
- Heuristica: $\tau \approx 0{,}3\times$ radio promedio de celula.

Conexion con el pipeline FPM-CubeSat

`tools/tune_onion.py:segment` implementa exactamente este pipeline. El slider `Marker dist` es τ. Si te juntan celulas grandes vecinas: subi τ. Si te parte celulas individuales en pedacitos: bajalo.

Ejercicios

1. Generar una imagen sintetica con 4 circulos parcialmente solapados. Aplicar Otsu y luego watershed con seeds detectados via distance transform. ¿Se separan?
2. Variar τ y reportar cuantos circulos detecta vs el ground truth.
3. Implementar la *distance transform* con doble pasada (forward + backward) en NumPy. Comparar con `cv2.distanceTransform`.

Capítulo 10

Forma, momentos y descriptores

10.1. Por que importa

Una vez que tenes la mascara de cada objeto, queres *describirlo* con un vector compacto: area, perimetro, redondez, orientacion. Estos descriptores son la entrada de cualquier clasificador clasico y la base para el control de calidad post-segmentacion.

10.2. Momentos de imagen

Definicion 10.1 (Momentos crudos)**.** *Para una funcion $I(x, y) \geq 0$:*

$$M_{pq} = \sum_{x,y} x^p y^q I(x, y). \tag{10.1}$$

Casos especiales:

- M_{00}: *masa total* (= area si I es binaria).
- Centroide: $\bar{x} = M_{10}/M_{00}$, $\bar{y} = M_{01}/M_{00}$.

Momentos centrales (invariantes a traslacion):

$$\mu_{pq} = \sum_{x,y} (x - \bar{x})^p (y - \bar{y})^q I(x, y). \tag{10.2}$$

Momentos de Hu (invariantes a traslacion, escala y rotacion): siete combinaciones polinomicas de los μ_{pq} normalizados. Util para reconocimiento de formas independiente de la pose.

10.3. Descriptores de contorno

Dado un contorno C (lista ordenada de puntos) de un objeto:

- **Area**: $|R|$ del interior.
- **Perimetro**: $\sum_i \|p_{i+1} - p_i\|$.
- **Circularidad**: $4\pi A/P^2$. Vale 1 para circulo perfecto, < 1 para formas alargadas o irregulares.
- **Aspect ratio**: razon entre los ejes mayor y menor del bounding box rotado o de la elipse ajustada.
- **Solidez**: $A/A_{\text{convex hull}}$. Mide concavidades.
- **Excentricidad**: $\sqrt{1 - (b/a)^2}$ donde a, b son los semiejes de la elipse ajustada.

10.3.1. Ajuste de elipse

Dado un conjunto de puntos del contorno, encontrar la elipse de minimo error cuadratico. Solucion cerrada via SVD de la matriz $D = [x^2, xy, y^2, x, y, 1]$.

10.4. Cadenas de Freeman

Una representacion compacta del contorno: en cada paso, codifica la direccion (1 de 8) al siguiente pixel. Cadena resultante es facil de comparar entre objetos rotados.

Conexion con el pipeline FPM-CubeSat

`pipeline/measurement.py:measure_cells` calcula area, perimetro, circularidad, aspect ratio y solidez para cada contorno detectado. Se basa en `cv2.contourArea`, `cv2.arcLength`, `cv2.fitEllipse` y `cv2.convexHull`. Tu `compute_summary` agrega estadisticas globales: media, desvio, min, max.

Ejercicios

1. Implementar el calculo de area de un poligono dado por sus vertices (formula del shoelace).
2. Calcular los momentos M_{00}, M_{10}, M_{01} para una mascara binaria. Verificar que reproducen el centroide.
3. Sobre tus resultados de segmentacion: graficar histogramas de area, circularidad, AR. ¿Que percentiles detectan outliers obvios?
4. Implementar momentos de Hu para 5 formas: rectangulo, cuadrado, triangulo, circulo, estrella. Verificar invariancia a rotacion.

Parte III

El espacio de Fourier — la otra mitad del mundo

Capítulo 11

Transformada de Fourier 2D

11.1. Por que importa

La FT es la otra cara de la moneda: cualquier operacion lineal e invariante a traslaciones (incluyendo *toda* la convolucion) es *simple* en el dominio de la frecuencia. La fisica de tu microscopio FPM *vive* en este espacio: la apertura del objetivo es un filtro paso bajo en frecuencia. Sin Fourier, FPM es magia negra; con Fourier, es un teorema.

11.2. Definicion

Definicion 11.1 (FT 2D continua)**.**

$$\hat{I}(\xi,\eta) = \mathcal{F}\{I\}(\xi,\eta) = \iint_{\mathbb{R}^2} I(x,y)e^{-2\pi i(\xi x+\eta y)}\,dx\,dy. \tag{11.1}$$

Definicion 11.2 (FT 2D discreta (DFT))**.** *Para una imagen $W \times H$:*

$$\hat{I}[u,v] = \sum_{x=0}^{W-1}\sum_{y=0}^{H-1} I[x,y]\exp\left(-2\pi i\left(\frac{ux}{W}+\frac{vy}{H}\right)\right). \tag{11.2}$$

La FFT calcula la DFT en $O(WH\log(WH))$.

11.2.1. Magnitud y fase

$\hat{I}$ es complejo: lo escribimos $\hat{I} = |\hat{I}|e^{i\phi}$.

- $|\hat{I}|$: *magnitud*. Indica cuanta energia hay en cada frecuencia.
- ϕ: *fase*. Codifica la *posicion* de las estructuras.

Hecho fundamental: la fase contiene la mayor parte de la informacion estructural. Si reemplazas la fase de una imagen por la de otra, ves la *otra* imagen. Por eso aprender que la fase importa mas que la magnitud es una intuicion clave.

11.3. Propiedades fundamentales

$$\text{Linealidad}: \quad \mathcal{F}\{\alpha I_1 + \beta I_2\} = \alpha \hat{I}_1 + \beta \hat{I}_2 \tag{11.3}$$

$$\text{Traslacion}: \quad I(x - x_0, y - y_0) \leftrightarrow \hat{I}(\xi, \eta) e^{-2\pi i(\xi x_0 + \eta y_0)} \tag{11.4}$$

$$\text{Convolucion}: \quad I * K \leftrightarrow \hat{I} \cdot \hat{K} \quad (\text{Teorema } 3.14) \tag{11.5}$$

$$\text{Parseval}: \quad \sum_{x,y} |I[x,y]|^2 = \frac{1}{WH} \sum_{u,v} |\hat{I}[u,v]|^2 \tag{11.6}$$

Parseval dice: la energia total se preserva entre dominios.

11.4. Filtrado en frecuencia

11.4.1. Receta general

1. Calcular $\hat{I} = \text{FFT}(I)$.
2. Centrar el espectro con `fftshift`.
3. Multiplicar por una mascara $M(\xi, \eta)$.
4. Inverse fftshift.
5. Calcular $I' = \text{IFFT}(\hat{I} \cdot M)$.

11.4.2. Filtro paso bajo ideal (low-pass)

$$M(\xi, \eta) = \begin{cases} 1 & \sqrt{\xi^2 + \eta^2} \leq \rho_0 \\ 0 & \text{si no} \end{cases} \tag{11.7}$$

Atrapa solo frecuencias bajas $\rightarrow$ imagen suavizada. **Problema**: el corte abrupto introduce *ringing* (oscilaciones tipo Gibbs). Mejor usar un Butterworth o Gaussiano:

$$M_G(\xi, \eta) = \exp\left(-\frac{\xi^2 + \eta^2}{2\sigma^2}\right). \tag{11.8}$$

11.4.3. Filtro paso alto

$M_{\text{hp}} = 1 - M_{\text{lp}}$. Resalta detalles finos.

11.4.4. Filtro paso banda

Anillo entre dos radios. Util para texturas a una escala especifica.

```
import numpy as np
F = np.fft.fft2(img.astype(np.float32))
F_shift = np.fft.fftshift(F)
H, W = img.shape
cy, cx = H // 2, W // 2
yy, xx = np.indices((H, W))
rho = np.sqrt((xx - cx)**2 + (yy - cy)**2)
mask = np.exp(-rho**2 / (2 * 30**2))   # Gaussian LP, sigma=30 pix
F_filtered = F_shift * mask
img_lp = np.real(np.fft.ifft2(np.fft.ifftshift(F_filtered)))
```

Listing 11.1: Filtrado paso bajo via FFT.

Conexion con el pipeline FPM-CubeSat

`pipeline/fpm_reconstruction.py` aplica filtros circulares en el dominio de frecuencia para emular la apertura de un objetivo. Cada captura del scan multi-angulo corresponde a una banda diferente del espectro. La reconstruccion FPM *ensambla* estas bandas en un espectro de mayor cobertura $\rightarrow$ super-resolucion.

Ejercicios

1. Calcular FFT y mostrar magnitud (en escala log) de tu imagen de cebolla. Identificar las frecuencias dominantes.
2. Aplicar paso bajo con $\sigma = 5, 20, 50$ y comparar con un Gaussian blur del mismo σ en espacio. Verificar el teorema de la convolucion.
3. Reemplazar la fase de la imagen A con la de la imagen B. Reconstruir. ¿Cual aparece dominante?
4. Implementar un filtro *notch* para eliminar una frecuencia especifica (util si tu imagen tiene un patron periodico no deseado).

Capítulo 12

Optica de Fourier para microscopia

12.1. El microscopio como sistema lineal

Bajo aproximaciones de optica paraxial e iluminacion espacialmente coherente, el microscopio se modela como un sistema lineal e invariante. La salida es:

$$I_{\text{img}}(x, y) = (I_{\text{obj}} * \text{PSF})(x, y), \tag{12.1}$$

donde la *point spread function* (PSF) describe la respuesta del sistema a una fuente puntual.

En frecuencia:

$$\hat{I}_{\text{img}} = \hat{I}_{\text{obj}} \cdot \text{OTF}, \qquad \text{OTF} = \mathcal{F}\{\text{PSF}\}. \tag{12.2}$$

La *magnitud* de la OTF se llama **MTF** (Modulation Transfer Function) y caracteriza el rango de frecuencias que el microscopio "deja pasar".

12.2. Apertura numerica y limite de difraccion

Definicion 12.1 (Apertura numerica)**.**

$$\text{NA} = n \sin \theta_{\text{máx}}, \tag{12.3}$$

con n el indice de refraccion del medio y $\theta_{\text{máx}}$ el angulo maximo de aceptacion del objetivo.

El *limite de Abbe* establece la resolucion espacial:

$$d_{\text{mín}} = \frac{\lambda}{2\,\text{NA}}. \tag{12.4}$$

Para luz visible ($\lambda \approx 500$ nm) y NA = 0,1 (lensless tipico): $d_{\text{mín}} \approx 2{,}5$ µm. Por eso una sola captura no resuelve celulas pequeñas.

12.3. Iluminacion oblicua = acceso a frecuencias altas

Cuando iluminas la muestra con luz que llega a un angulo θ desde la normal, en el dominio de frecuencia eso *traslada* el espectro de la muestra:

$$\hat{I}_{\text{obj}}(\xi - \xi_0, \eta - \eta_0) \tag{12.5}$$

con ξ_0, η_0 proporcionales a $\sin \theta$. Frecuencias que estaban afuera de la apertura (limite de Abbe) ahora caen *dentro*. Capturas diferentes con diferentes angulos te dan diferentes *bandas* del espectro real del objeto.

Esa es la idea de FPM.

Capítulo 13

Fourier Ptychographic Microscopy (FPM)

13.1. El problema

Un objetivo de bajo NA (lente o lensless) tiene gran campo de vision pero baja resolucion. Un objetivo de alto NA tiene alta resolucion pero campo de vision microscopico. ¿Podemos tener ambos?

13.2. La idea de FPM

Adquirir N imagenes con *iluminacion* desde diferentes angulos (en tu CubeSat: una matriz LED de OLED 5×5 o 11×11). Cada imagen captura una banda diferente del espectro del objeto, todas filtradas por el mismo objetivo de bajo NA.

Despues, reconstruir el espectro completo combinando todas las bandas → imagen sintetizada con NA efectivo $\sim N\times$ mayor.

13.3. Algoritmo de reconstruccion

Iterativo, tipo Gerchberg-Saxton:

1. Inicializar el espectro estimado $\hat{O}$ (zero-padded, alta resolucion).
2. Para cada captura I_n con angulo (θ_n, ϕ_n):
   a. Recortar la sub-banda de $\hat{O}$ correspondiente a la apertura desplazada por (ξ_{0n}, η_{0n}).
   b. Hacer IFFT → imagen estimada de baja resolucion.
   c. *Actualizar fase*: reemplazar la magnitud de la imagen estimada por $\sqrt{I_n}$ (la magnitud medida).
   d. FFT y reinsertar en la sub-banda de $\hat{O}$.
3. Repetir hasta convergencia.

Es exactamente *phase retrieval*: usamos magnitud medida para recuperar fase no medida, usando la consistencia entre los N submuestreos.

13.3.1. Cuantos angulos necesitas

La cobertura del espectro reconstruido es la union de los discos de apertura desplazados. Para cobrir un disco mayor de NA efectivo $N \cdot \mathrm{NA}$ necesitas un solapamiento de $\sim 60\,\%$ entre angulos consecutivos.

Conexion con el pipeline FPM-CubeSat

Tu `config.yaml` en la seccion `fpm` controla los parametros: `upscale_factor` (cuanto crece la resolucion sintetizada), `max_iters` (iteraciones del algoritmo), `na_obj` (apertura del sistema, lensless 0.05–0.15), `distance_mm` (distancia OLED-sensor). `pipeline/fpm_reconstruction.py` implementa el bucle de phase retrieval. Tu `cubesat/capture.py` captura los N angulos automaticamente en la Pi.

Ejercicios

1. Simular FPM en 1D: generar una señal compuesta por 3 frecuencias, capturarla con 5 ventanas de paso banda solapadas, reconstruir.
2. En tu codigo: cambiar `upscale_factor` de 2 a 4. ¿Cuanto tarda? ¿Mejora la imagen?
3. Variar `na_obj` en $\{0{,}05, 0{,}10, 0{,}15\}$. ¿Cuando converge mejor?
4. Implementar la reconstruccion con pena de regularizacion (L2 sobre el gradiente del modulo) para reducir ruido.

Parte IV

Multi-imagen — registracion y super-resolucion

Capítulo 14

Registracion de imagenes

Antes de combinar varias capturas (como en FPM o multi-frame SR) hay que *alinearlas*. Tipos de transformaciones tipicas:

Transformacion	**DOF**	**Modela**
Traslacion	2	shift puro entre frames
Rigida	3	traslacion + rotacion
Similar	4	rigida + escala uniforme
Afin	6	rigida + escala + cizalla (sensor inclinado)
Homografia	8	camara movida con plano comun (tablero)

14.0.1. Phase correlation

Para traslacion pura, la *phase correlation* usa la propiedad de traslacion en Fourier:

$$\frac{\hat{I}_1^* \cdot \hat{I}_2}{|\hat{I}_1^* \cdot \hat{I}_2|} = e^{-2\pi i(\xi\Delta x + \eta\Delta y)}, \tag{14.1}$$

cuya IFFT es un impulso en $(\Delta x, \Delta y)$. Robusta y rapida.

14.0.2. ECC (Enhanced Correlation Coefficient)

Optimizacion iterativa que minimiza el error normalizado entre las dos imagenes parametrizando una transformacion (afin u homografia). Usa Lucas-Kanade. `cv2.findTransformECC`.

14.0.3. Feature-based: SIFT, ORB, AKAZE

Detectar puntos caracteristicos en ambas imagenes, hacer matching descriptor a descriptor, estimar la transformacion via RANSAC. Funciona con cambios de escala y rotacion grandes.

Conexion con el pipeline FPM-CubeSat

En FPM, las capturas a diferentes angulos pueden estar levemente desalineadas. Tu `config.yaml` tiene `align: true` en la seccion `fpm`, que activa registracion via ECC antes de la reconstruccion.

Capítulo 15

Super-resolucion

Single-image SR: imposible sin un prior. Las redes (Real-ESRGAN, SwinIR) aprenden ese prior de millones de pares LR-HR.

Multi-frame SR: N imagenes con shifts *sub-pixel* contienen informacion de frecuencia que cualquier imagen sola no tiene. Algoritmo basico:

1. Estimar shifts sub-pixel (phase correlation).
2. Inicializar imagen HR.
3. Iterativamente: simular las N LR a partir del estimado HR, comparar con las medidas, retropropagar el error.

FPM es un caso particular donde los "shifts" estan en *frecuencia*, no en espacio.

Parte V

Aprendizaje profundo en vision

Capítulo 16

Del perceptron a la CNN

16.1. El perceptron

Un perceptron toma un vector de entrada $\mathbf{x}$, un peso $\mathbf{w}$, un bias b, y una no linealidad σ:

$$y = \sigma(\mathbf{w}^\top \mathbf{x} + b). \tag{16.1}$$

Apilar varias capas $\rightarrow$ MLP. Aproxima cualquier funcion continua (Cybenko, 1989) pero *no* aprovecha la estructura espacial de una imagen: trata cada pixel como una variable independiente.

16.2. La capa convolucional

Reemplazamos la multiplicacion matricial por una convolucion:

$$y[i, j, c] = \sum_{c'} \sum_{m,n} K[m, n, c', c] \cdot x[i + m, j + n, c'] + b[c]. \tag{16.2}$$

Beneficios:

- **Compartir pesos**: el mismo kernel se aplica en toda la imagen. Equivariante a traslacion.
- **Locality**: cada salida depende de un parche local (*receptive field*).
- **Parametros** $O(k^2cc')$ en lugar de $O(WHcc')$ de un fully-connected.

16.3. Pooling y receptive field

Max pooling reduce resolucion tomando el maximo de cada sub-region 2×2. Aumenta el receptive field de las capas siguientes y da invariancia local.

El *receptive field* de una neurona en la capa L es la region de la imagen de entrada que influye en su valor. Crece con la profundidad (cada capa con kernel k y stride s multiplica el RF anterior).

16.4. Backpropagation

El entrenamiento minimiza una loss $\mathcal{L}$ ajustando los pesos por gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}. \tag{16.3}$$

Backpropagation calcula $\nabla_{\mathbf{w}} \mathcal{L}$ aplicando la regla de la cadena en orden reverso por las capas. Frameworks como PyTorch/TF lo automatizan.

Capítulo 17

U-Net y segmentacion semantica

17.1. La arquitectura U-Net

Ronneberger et al. (2015) propusieron U-Net: una red en forma de "U" con *encoder* (downsampling) y *decoder* (upsampling), conectados por *skip connections* a cada nivel.

- El *encoder* reduce resolucion espacial pero aumenta canales: aprende features de nivel alto.
- El *decoder* reconstruye resolucion espacial: convierte features en una mascara densa.
- Las *skip connections* concatenan features finos del encoder con los del decoder en la misma escala $\rightarrow$ preservan localizacion exacta.

Es *el* modelo de referencia en microscopia y se popularizo justamente porque entrena con pocos datos (data augmentation aggressive sobre las pocas mascaras anotadas).

17.2. Loss functions para segmentacion

Cross-entropy pixel-wise: estandar para clasificacion por pixel.
Dice loss:

$$\mathcal{L}_{\text{dice}} = 1 - \frac{2\sum p_i g_i}{\sum p_i + \sum g_i}. \tag{17.1}$$

Robusta cuando hay desbalance entre clases (mas fondo que objeto).

Focal loss: pondera los pixeles dificiles. Util si una pequeña fraccion de pixeles concentra el error.

Capítulo 18

Modelos especificos para microscopia

18.1. Cellpose

Pachitariu & Stringer (2021). Una U-Net entrenada con datasets diversos de microscopia que predice *flujos* 2D apuntando al centro de cada celula. Despues, integra esos flujos con un metodo tipo *watershed*: cada pixel sigue su flujo hasta caer en un basin, que define una instancia.

Cuando usar: imagenes de fluorescencia con celulas de cualquier morfologia. Para bright-field como tu cebolla: posible pero da resultados mediocres con el modelo `cyto3` default. Necesita fine-tune.

18.2. StarDist

Schmidt & Weigert (2018). Predice por pixel un conjunto de *distancias radiales* a la frontera del objeto en K direcciones. Asume cada objeto es *star-convex*: el centroide ve toda la frontera sin obstaculos. Cierto para celulas de fluorescencia y *aproximadamente* cierto para celulas epidermicas.

Modelos preentrenados:

- `2D_versatile_fluo`: fluorescencia. Mal fit para bright-field.
- `2D_versatile_he`: histologia H&E. *Mejor opcion para cebolla* en bright-field.
- `2D_paper_dsb2018`: dataset DSB de Kaggle, mas variado.

18.3. Real-ESRGAN

GAN para super-resolucion. Genera detalles fotorealistas de alta frecuencia que no estan en la imagen LR. **Cuidado**: puede inventar detalles. Para microscopia cuantitativa esto es problematico — las "celulas" agregadas por la red podrian ser alucinaciones.

Capítulo 19

Validacion y metricas

19.1. Para clasificacion

Accuracy: fraccion correctamente clasificada. Engañosa con clases desbalanceadas.
Precision: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. Cuantos de los positivos predichos son correctos.
Recall: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. Cuantos de los positivos reales detectaste.
F1: media armonica de precision y recall.

19.2. Para segmentacion semantica

IoU (Intersection over Union, Jaccard):

$$\mathrm{IoU} = \frac{|A \cap B|}{|A \cup B|}. \tag{19.1}$$

Cero si no hay solapamiento, uno si las mascaras coinciden exactamente.

Dice:

$$\mathrm{Dice} = \frac{2|A \cap B|}{|A| + |B|} = \frac{2\mathrm{IoU}}{1 + \mathrm{IoU}}. \tag{19.2}$$

19.3. Para segmentacion por instancias

Average Precision (AP) a un IoU threshold t: cada celula predicha se "matchea" con una real si $\mathrm{IoU} > t$. Se calcula precision/recall y se promedia.

El estandar moderno reporta AP@[0,5:0,05:0,95]: el promedio sobre 10 thresholds.

19.4. Por que no tener metricas es no tener pipeline

Si no medis con un numero objetivo, no podes:

- Comparar dos versiones del codigo.
- Saber si una optimizacion ayudo.
- Detectar regresiones cuando agregas features.
- Justificar decisiones (¿uso CLAHE? ¿uso Cellpose?).

Tu pipeline necesita: un set pequeño (5–10) de imagenes con *mascaras anotadas a mano*, un script que corra el pipeline sobre ellas y reporte IoU, Dice, AP. Con eso, cualquier mejora se vuelve cuantificable.

Parte VI

Vanguardia

Capítulo 20

Self-supervised learning

20.1. La idea

Entrenar redes sin etiquetas usando *tareas pretexto*: predecir partes de la entrada a partir de otras partes. Ejemplo: dado un parche enmascarado, predecir su contenido.

20.1.1. Noise2Void (N2V)

Krull et al. (2019). Entrenar una red para predecir cada pixel usando todos los *otros* (*blind spot*). Si el ruido es independiente entre pixeles, la red no puede aprenderlo y converge a la media de la señal sin ruido.

Lo usas para denoising sin ground truth limpio. `pipeline/ai_enhance.py:run_n2v` es una implementacion minima.

20.1.2. Contrastive learning (SimCLR, MoCo, DINO)

Generar dos vistas de la misma imagen (data augmentation), entrenar la red para que sus representaciones sean cercanas, y lejanas a las de otras imagenes.

Capítulo 21

Foundation models

21.1. SAM — Segment Anything

Meta AI (2023). Una red gigante (ViT) entrenada con 11M imagenes y 1B mascaras. Recibe la imagen y un prompt (punto, caja, mascara aproximada) y devuelve mascaras de calidad. *Zero-shot*: no necesita fine-tune para dominios nuevos.

Para tu cebolla: prometedor, pero los prompts manuales no escalan. Hay variantes (SAM-2 video, MedSAM) para uso medico.

21.2. DINO / DINOv2

Self-supervised que entrena con augmentations + teacher-student distillation. Genera features que capturan estructura sin etiquetas. Excelentes representaciones para downstream tasks.

21.3. Modelos de difusion (Stable Diffusion, etc.)

Generan imagenes a partir de ruido invirtiendo iterativamente un proceso de difusion. Usados para super-resolucion (SR3, ResShift), restauracion (DDRM, DiffPIR), y generacion condicional. Para microscopia, todavia con poca adopcion porque pueden alucinar.

Capítulo 22

NeRF y 3D Gaussian Splatting

22.1. NeRF (Neural Radiance Fields)

Mildenhall et al. (2020). Una MLP que recibe (x, y, z, θ, ϕ) y devuelve radiancia (R, G, B) + densidad. Para renderizar un pixel se integra a lo largo del rayo. Permite reconstruccion 3D fotorealista a partir de N fotos.

Aplicaciones en microscopia: tomografia computacional reciente.

22.2. Gaussian Splatting

Kerbl et al. (2023). Reemplaza la representacion implicita de NeRF por una nube de gaussianas 3D. Mucho mas rapido (60+ fps en GPU). Estado del arte actual para reconstruccion volumetrica.

Apendice A — Matematica de bolsillo

22.3. Algebra lineal

Producto interno: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$.
Norma: $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$.
Autovalores y autovectores: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. La descomposicion espectral $\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1}$.
SVD: $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$. Funciona para matrices no cuadradas. Base de PCA, pseudo-inversa, baja-rango.

22.4. Calculo

Gradiente: $\nabla f = (\partial_x f, \partial_y f, \dots)$. Apunta en la direccion de mayor crecimiento.
Hessiana: matriz de segundas derivadas. Sus autovalores describen curvatura.
Regla de la cadena: $\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$. Base de backprop.

22.5. Probabilidad

Distribucion gaussiana 1D: $\mathcal{N}(\mu, \sigma^2)$ con densidad $\frac{1}{\sigma\sqrt{2\pi}} \exp(-(x-\mu)^2/2\sigma^2)$.
Teorema de Bayes: $P(A|B) = P(B|A)P(A)/P(B)$.
Esperanza, varianza, covarianza: estadisticos de primer y segundo orden.

Apendice B — Hoja de ruta de aprendizaje

22.6. Plan de 6 meses (10 horas/semana)

Mes 1 — Fundamentos. Capitulos 1–5. Implementar todo en NumPy, sin OpenCV. Resultado: entendes que es un pixel, una convolucion, una operacion morfologica.

Mes 2 — Bordes y segmentacion clasica. Capitulos 6–10. Implementar Canny, Frangi, watershed. Resultado: tu segmentador OpenCV competitivo, sin IA.

Mes 3 — Fourier. Capitulos 11–13. Entender FFT 2D, filtrado en frecuencia, optica de Fourier, FPM. Resultado: podes leer un paper de FPM y modificar tu reconstruccion.

Mes 4 — Multi-imagen y registracion. Implementar phase correlation, ECC, super-resolucion multi-frame.

Mes 5 — Deep learning. Entrenar un perceptron, CNN, U-Net mini. Aplicar a celulas. Cuantificar IoU.

Mes 6 — Vanguardia. Leer papers actuales (Cellpose, StarDist, SAM). Reproducir resultados. Aportar al pipeline.

22.7. Recursos imprescindibles

Libros

Gonzalez & Woods, *Digital Image Processing* (4ed). Szeliski, *Computer Vision: Algorithms and Applications* (gratis online). Goodfellow, Bengio & Courville, *Deep Learning* (gratis online).

Cursos en video

Shree Nayar, *First Principles of Computer Vision* (Columbia, YouTube, 60 lecturas gratis). Stanford CS231n (Karpathy, Li). MIT 6.819 *Advances in Computer Vision* (Torralba).

Papers fundacionales

Lowe, *SIFT* (1999). Canny, *A Computational Approach to Edge Detection* (1986). Frangi et al., *Multiscale vessel enhancement filtering* (1998). Ronneberger, *U-Net* (2015). Krizhevsky, *AlexNet* (2012). Vaswani, *Attention is All You Need* (2017).

Para microscopia y FPM

Zheng et al., *Wide-field, high-resolution Fourier ptychographic microscopy* (Nature Photonics, 2013). Pachitariu & Stringer, *Cellpose 2.0* (Nature Methods, 2022). Schmidt et al., *StarDist* (MICCAI, 2018).

22.8. Prediccion

Si seguis este plan con disciplina, en 6 meses vas a estar leyendo papers de CVPR/ICCV sin sentir que te falta vocabulario, y vas a poder modificar cualquier modulo de tu pipeline (clasico

o IA) entendiendo *por que* funciona.

Bibliografia

1. Gonzalez RC, Woods RE. *Digital Image Processing*. 4ed. Pearson, 2018.
2. Szeliski R. *Computer Vision: Algorithms and Applications*. 2ed. Springer, 2022. https://szeliski.org/Book/
3. Goodfellow I, Bengio Y, Courville A. *Deep Learning*. MIT Press, 2016. https://www.deeplearningbook.org/
4. Bhandari A, Kadambi A, Raskar R. *Computational Imaging*. MIT Press, 2022.
5. Canny J. A Computational Approach to Edge Detection. *IEEE TPAMI* 8(6), 1986.
6. Frangi AF, Niessen WJ, Vincken KL, Viergever MA. Multiscale vessel enhancement filtering. *MICCAI*, 1998.
7. Beucher S, Meyer F. The morphological approach to segmentation: the watershed transformation. *Mathematical Morphology in Image Processing*, 1993.
8. Otsu N. A Threshold Selection Method from Gray-Level Histograms. *IEEE TSMC* 9(1), 1979.
9. Buades A, Coll B, Morel JM. A non-local algorithm for image denoising. *CVPR*, 2005.
10. Krull A, Buchholz TO, Jug F. Noise2Void — Learning Denoising from Single Noisy Images. *CVPR*, 2019.
11. Zheng G, Horstmeyer R, Yang C. Wide-field, high-resolution Fourier ptychographic microscopy. *Nature Photonics* 7, 2013.
12. Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 2015.
13. Stringer C, Wang T, Michaelos M, Pachitariu M. Cellpose: a generalist algorithm for cellular segmentation. *Nature Methods* 18, 2021.
14. Schmidt U, Weigert M, Broaddus C, Myers G. Cell Detection with Star-Convex Polygons. *MICCAI*, 2018.
15. Kirillov A, et al. Segment Anything. *ICCV*, 2023.
16. Mildenhall B, et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. *ECCV*, 2020.
17. Kerbl B, Kopanas G, Leimkühler T, Drettakis G. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *SIGGRAPH*, 2023.